ISPARTA UYGULAMALI BİLİMLER ÜNİVERSİTESİ

ELEKTRİK ELEKTRONİK MÜHENDİSLİĞİ

MAKİNE ÖĞRENMESİNE GİRİŞ VE MODEL DEĞERLENDİRME

Makine Öğrenmesine Giriş

HAZIRLAYAN

Sudenaz Serin-2322705049

1. Giriş

Günümüzde veri miktarının ve işlem gücünün hızla artması klasik kural tabanlı programlamanın yetersiz kaldığı noktalar için makine öğrenmesini çözüm aracı haline getirmiştir. Ancak başarılı bir sistem tasarlamak sadece bir algoritma seçmekten ibaret olmayıp verinin toplanmasından özelliklerin çıkarılmasına kadar tüm sürecin uçtan uca bir boru hattı (pepeline) olarak yönetilmesi sistemin asıl başarısını belirlemektedir [1]. Bu süreçteki temel mühendislik hedefi modelin eğitim verisini sadece ezberlemesi değil daha önce karşılaşmadığı yeni verilere karşı başarılı bir genelleme yapabilmesidir. Bu genelleme yeteneğini kazanırken en kritik problem olan Yanlılık (Bias) ve Varyans dengesi iyi kurulamazsa model ya veriyi hiç kavrayamaz ya da rastgele gürültüleri ezberleyerek gerçek dünya verilerinde başarısız olur [2].

Bu çalışma kapsamında, model kapasitesinin öğrenme yeteneği üzerindeki etkisini "tf_flowers" veri seti üzerinden deneysel olarak inceledim. Modelin performansını değerlendirirken sadece genel geçer "doğruluk" (accuracy) metriğine güvenmenin yanıltıcı olabileceği bilgisinden hareketle kesinlik, duyarlılık ve F1-skoru gibi metriklerin önemini teorik çerçevede ele aldım [3]. Kendi mini projemde ise farklı nöron sayılarına sahip iki uç mimari üzerinden elde ettiğim başarı ve kayıp eğrilerini analiz ederek literatürdeki bu temel kavramları somut bir şekilde gözlemlemeyi hedefledim.

2. Temel Kavramlar

Makine Öğrenmesi İş Akışı (ML Pipeline): Bir makine öğrenmesi projesi temel olarak verinin toplanmasından başlayıp modelin optimize edilmesine kadar uzanan döngüsel bir iş akışını kapsar. Bu süreçte öncelikle ham veri toplanarak ön işlemlerden geçirilir ve seçilen algoritmaya uygun formata dönüştürülür. Ardından eğitim aşaması başlar ve elde edilen sonuçlar çeşitli metriklerle değerlendirilir. Modelin test aşamasında beklenen performansı gösterememesi durumunda, alınan hatanın türüne göre analitik bir optimizasyon stratejisi belirlenmelidir. Bu noktada sorunun kaynağına inilerek; veri setinin genişletilmesi, verideki gürültülerin temizlenmesi veya model mimarisinin karmaşıklaştırılması gibi çözüm yollarından probleme en uygun olanı tercih edilir.

Bias ve Varyans İkilemi (Bias-Variance Trade-off): Modelin yaptığı toplam hata, temel olarak Yanlılık (Bias) ve Varyans (Variance) olmak üzere iki ana bileşene ayrılmaktadır. Yanlılık,

modelin problemi çok basite indirgemesi ve verideki karmaşık ilişkileri kavrayacak kapasiteye sahip olmaması durumunda ortaya çıkar. Literatürde bu durum "Eksik Öğrenme" (Underfitting) olarak adlandırılır. Varyans ise, modelin eğitim verisindeki rastgele dalgalanmalara ve gürültülere aşırı hassasiyet göstererek genel örüntüyü öğrenmek yerine veriyi birebir ezberlemesiyle sonuçlanan hatadır. Bu probleme de "Aşırı Öğrenme" (Overfitting) denilmektedir.

Makine öğrenmesindeki temel mühendislik problemi, bu iki hata kaynağı arasında ideal dengeyi bulabilmektir. Modelin karmaşıklığı örneğin sinir ağlarındaki katman derinliği veya polinom derecesi artırıldıkça yanlılık azalma eğilimi gösterirken, varyans artmaya başlar. Temel amaç, modelin eğitim verisini ezberlemeden, daha önce hiç karşılaşmadığı test verileri üzerinde en düşük hata oranını vereceği optimal karmaşıklık seviyesini yakalamaktır.

3. Yöntem / Model Açıklaması

Veri Bölme Stratejileri: Bir modelin veriyi ezberleme (overfitting) hatasına düşüp düşmediğini veya yeni verilere başarılı bir şekilde genelleme yapıp yapamadığını test edebilmek için, veri setinin eğitim öncesinde doğru bir stratejiyle bölünmesi gerekmektedir. Modelin daha önce karşılaşmadığı veriler üzerindeki performansını bağımsız bir şekilde ölçebilmek esastır. En temel yaklaşım veriyi Eğitim (Train), Doğrulama (Validation) ve Test olmak üzere üç ana gruba ayırmaktır. Ancak veri sayısının kısıtlı olduğu problemlerde, rastgele yapılan tek seferlik bir bölme işlemi modelin gerçek başarısı hakkında yanıltıcı sonuçlar doğurabilir.Bu sorunu aşmak için K-Katlı Çapraz Doğrulama (K-Fold Cross Validation) tekniğine başvurulur [4]. Bu yapıda veri seti, eşit boyutlara sahip "k" adet alt kümeye ayrılır. Algoritma "k-1" adet küme ile eğitilirken, dışarıda bırakılan o tek küme ile test edilir. Bu işlem, her alt küme tam olarak bir kez test verisi olarak kullanılana kadar "k" defa tekrarlanır. Döngü sonunda elde edilen "k" farklı hata skorunun ortalaması alınarak, istatistiksel olarak çok daha tutarlı ve güvenilir bir performans değerlendirmesi yapılmış olur.

Model Değerlendirme Metrikleri: Sınıflandırma algoritmalarının başarısını yalnızca "Doğruluk" (Accuracy) metriği üzerinden değerlendirmek, özellikle mühendislik problemlerinde kritik hatalara zemin hazırlayabilir. Bu noktada modelin karakteristik davranışını ve nerede hata yaptığını detaylı inceleyebilmek adına Karmaşıklık Matrisi tabanlı analizler kullanılır. Temel performans metrikleri; Doğru Pozitif (TP), Doğru Negatif (TN),

Yanlış Pozitif (FP) ve Yanlış Negatif (FN) değerleri üzerinden hesaplanmaktadır [3]. Formülasyonları ve kullanım amaçları aşağıda detaylandırılmıştır:

Doğruluk (Accuracy): Toplam veri setinde doğru yapılan tahminlerin genel oranıdır.

$$Accuracy = \frac{\mathrm{TP} + \mathrm{TN}}{\mathrm{TP} + \mathrm{FP} + \mathrm{TN} + \mathrm{FN}}$$

Kesinlik (Precision): Sistemin "pozitif" olarak işaretlediği tahminlerin gerçekte ne kadarının doğru olduğunu gösterir. Yanlış alarm vermenin (False Positive) operasyonel maliyetinin yüksek olduğu durumlarda dikkate alınır.

$$Precision = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}}$$

Duyarlılık / Geri Çağırma (Recall): Gerçekte "pozitif" olan durumların model tarafından yüzde kaç oranında başarıyla yakalandığını ifade eder. Gözden kaçırmanın (False Negative) çok riskli olduğu problemlerde en önemli metriktir.

$$Recall = \frac{TP}{\mathrm{TP} + \mathrm{FN}}$$

F1-Skoru: Kesinlik ve Duyarlılık değerlerinin harmonik ortalaması alınarak hesaplanır. Özellikle sınıflar arası dağılımın dengesiz olduğu problemlerde modelin gerçek performansını yansıtan en güvenilir ölçüttür.

$$F1 = 2 \times \frac{Precision \times Recall}{\mathrm{Precision} + \mathrm{Recall}}$$

4. Uygulama Alanları

Makine öğrenmesi algoritmaları, tıbbi görüntülemeden otonom araçlara ve güç elektroniğine kadar oldukça geniş bir mühendislik yelpazesinde kullanılmaktadır. Ancak bir modelin başarısını ölçecek metriklerin seçimi, doğrudan çözülmeye çalışılan problemin sahadaki doğasına göre belirlenmelidir.

Elektrik-elektronik mühendisliğinde kritik bir öneme sahip olan akustik ve titreşim analizi tabanlı sistem durum izleme çalışmaları, bu duruma net bir örnektir. Dönen makinelerden veya mekanik sistemlerden sensörler aracılığıyla toplanan veriler işlenerek, sistemin "sağlıklı" ya da "arızalı" çalışma durumları sınıflandırılır. Endüstriyel gerçeklikte mekanik sistemler ömürlerinin çok büyük bir kısmını sağlıklı bir şekilde geçirir ve arıza durumlarına oldukça nadir rastlanır. Bu durum, veri setinde ciddi bir sınıf dengesizliğine yol açar.

Sınıf dağılımının bu denli dengesiz olduğu bir projede, modelin performansını yalnızca "Doğruluk" (Accuracy) metriği ile değerlendirmek istatistiksel olarak yanıltıcıdır. Sistem, veriyi hiç analiz etmeden sürekli "makine sağlam" tahmini ürettiğinde bile %98 gibi yüksek bir doğruluk oranına ulaşabilir. Fakat gerçek bir mühendislik senaryosunda olası bir arızayı (False Negative) gözden kaçırmanın maliyeti; tüm sistemin çökmesi veya üretimin durmasıdır. Bu nedenle, bu tip projelerde modelin asıl başarısı, arızaları yakalama yeteneğini gösteren Duyarlılık (Recall) ve sahte alarmları (False Positive) da hesaba katarak dengeli bir ölçüm sunan F1-Skoru üzerinden analiz edilmelidir.

5. Yapılan Mini Proje Açıklaması

Projenin Amacı:

Bu çalışmada temel hedefim, bir yapay sinir ağı modelinin kapasitesinin nöron ve katman sayısı öğrenme ve genelleme yeteneği üzerindeki etkisini deneysel olarak incelemektir. Modelin veriyi gerçekten "öğrenip öğrenmediğini" veya sadece "ezberleyip ezberlemediğini" (overfitting) iki farklı uç senaryo üzerinden simüle etmeyi amaçladım.

Kullanılan Veri Seti:

Uygulamada, derin öğrenme literatüründe yaygın olarak kullanılan "tf_flowers" veri setini tercih ettim. Bu set; Papatya, Karahindiba, Gül, Ayçiçeği ve Lale olmak üzere toplam 5 farklı

çiçek sınıfına ait binlerce görsel içermektedir. Görüntüleri modelin daha hızlı işleyebilmesi için 64x64 piksel boyutuna indirgedim ve piksel değerlerini 0-1 arasına normalize ettim.

Kullanılan Yöntem ve Kodlamalar:

Projeyi Python dili ve TensorFlow/Keras kütüphanesini kullanarak geliştirdim. Eğitim sürecini sunumumda da belirttiğim "Makine Öğrenmesi Fabrikası" (ML Pipeline) iş akışına göre kurguladım. Veri setini %80 eğitim ve %20 doğrulama (validation) olarak ikiye böldüm. Kapasite etkisini görebilmek için şu iki farklı mimariyi tasarladım:

1. Küçük Kapasiteli Model: Sadece 1 gizli katman ve 16 nöron içeren, oldukça düşük kapasiteli bir ağ kurguladım.
2. Büyük Kapasiteli Model: 2 gizli katmanda toplam 1024 (512+512) nöron bulunan, yaklaşık 6.5 milyon parametreye sahip çok yüksek kapasiteli bir yapı oluşturdum.

Elde Edilen Sonuçlar ve Değerlendirme:

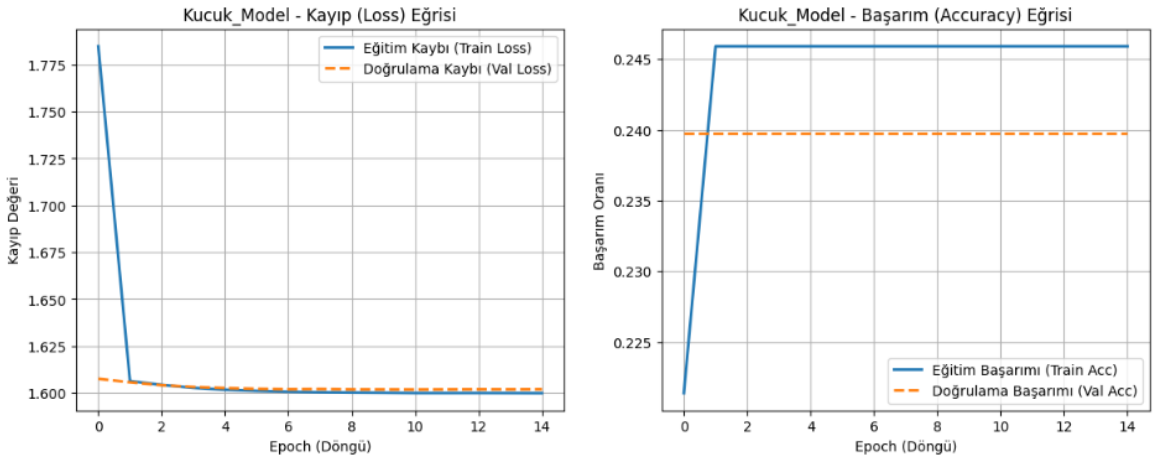
```
small_model = models.Sequential([
  layers.Rescaling(1./255, input_shape=(img_height, img_width, 3)), # Piksel değerlerini 0-1 arasına çekme
  layers.Flatten(),
  layers.Dense(16, activation='relu'), # Çok düşük nöron sayısı
  layers.Dense(5) # 5 farklı çiçek sınıfı için çıkış katmanı
], name="Kucuk_Model")

small_model.compile(optimizer='adam',
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=['accuracy'])
small_model.summary()
```

Model: "Kucuk_Model"

Layer (type)	Output Shape	Param #
rescaling_2 (Rescaling)	(None, 64, 64, 3)	0
flatten_2 (Flatten)	(None, 12288)	0
dense_7 (Dense)	(None, 16)	196,624
dense_8 (Dense)	(None, 5)	85

Total params: 196,709 (768.39 KB)
Trainable params: 196,709 (768.39 KB)
Non-trainable params: 0 (0.00 B)

Şekil 1: Küçük Kapasiteli Modelin Katman Yapısı ve Parametre Özet Tablosu.



Şekil 2: Küçük Kapasiteli Modelin (16 Nöron) Eğitim ve Doğrulama Başarı (Accuracy) Eğrileri.

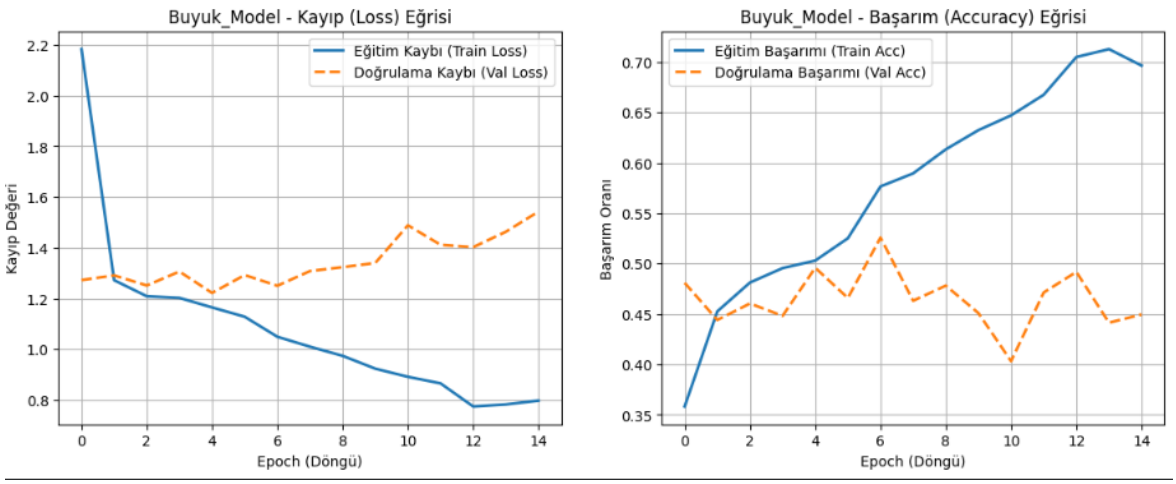
```
large_model = models.Sequential([
  layers.Rescaling(1./255, input_shape=(img_height, img_width, 3)),
  layers.Flatten(),
  layers.Dense(512, activation='relu'), # Aşırı yüksek nöron sayısı
  layers.Dense(512, activation='relu'),
  layers.Dense(5)
], name="Buyuk_Model")

large_model.compile(optimizer='adam',
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=['accuracy'])
large_model.summary()
```

Model: "Buyuk_Model"

Layer (type)	Output Shape	Param #
rescaling_3 (Rescaling)	(None, 64, 64, 3)	0
flatten_3 (Flatten)	(None, 12288)	0
dense_9 (Dense)	(None, 512)	6,291,968
dense_10 (Dense)	(None, 512)	262,656
dense_11 (Dense)	(None, 5)	2,565

Total params: 6,557,189 (25.01 MB)
Trainable params: 6,557,189 (25.01 MB)
Non-trainable params: 0 (0.00 B)

Şekil 3: Büyük Kapasiteli Modelin (1024 Toplam Nöron) Katman Yapısı ve Parametre Özet Tablosu.



Şekil 4: Büyük Kapasiteli Modelin Eğitim ve Doğrulama Süreçlerine Ait Kayıp (Loss) ve Başarım (Accuracy) Grafikleri.

1. Küçük Kapasiteli Modelin Analizi: Şekil 1'de mimari detayları görülen 16 nöronluk modelin, çiçek görsellerindeki karmaşık öznitelikleri çıkarabilecek matematiksel derinliğe sahip olmadığı gözlemlenmiştir. Şekil 2'deki başarım eğrileri incelendiğinde, eğitim ve doğrulama başarılarının %24 bandında çakılı kaldığı görülmektedir. 5 sınıflı bir problemde bu oran, modelin neredeyse rastgele tahmin yürüttüğünü göstermektedir. Bu durum literatürde, modelin veriyi temsil edemeyecek kadar basit kalması olarak tanımlanan "Eksik Öğrenme" (Underfitting) teşhisinin somut bir örneğidir.

2. Büyük Kapasiteli Modelin Analizi: Şekil 3'te sunulan yaklaşık 6.5 milyon parametreli devasa mimarinin ise tam tersi bir problem yaşadığı tespit edilmiştir. Şekil 4'teki grafikler analiz edildiğinde; eğitim başarısının hızla artarak %70'lere ulaşmasına rağmen, doğrulama başarısının %45-50 bandında tıkanıp dalgalandığı görülmektedir. Özellikle doğrulama kaybının (validation loss) belirli bir noktadan sonra düşmek yerine yukarı yönlü sekmesi, modelin genel örüntüleri öğrenmek yerine eğitim verisindeki gürültüleri birebir ezberlediğini kanıtlamaktadır. Bu durum, modelin genelleme yeteneğini kaybederek "Aşırı Öğrenme" (Overfitting) durumuna geçtiğini göstermektedir.

Her iki senaryodan elde edilen bulgular, başarılı bir makine öğrenmesi sisteminin temelinde yatan Yanlılık (Bias) **ve** Varyans dengesinin önemini doğrulamaktadır. Küçük modelde yüksek yanlılık (bias), büyük modelde ise yüksek varyans hatasıyla karşılaşılmıştır. Sonuç olarak, en yüksek performansın ancak veri setinin karmaşıklığına uygun optimum kapasitedeki mimariyle yakalanabileceği sonucuna varılmıştır.

6. Sonuç

Bu çalışmada, bir makine öğrenmesi sisteminin başarısının sadece modelin büyüklüğüne bağlı olmadığını uygulamalı olarak ispatladım. Küçük modelin veriyi kavrayamadığı, büyük modelin ise veriyi gereğinden fazla ezberlediği (overfitting) uç noktaları inceleyerek, en yüksek performansın "optimum karmaşıklık" noktasında yattığını anladım.

Süreçten çıkardığım en kritik mühendislik dersi, bir modelin başarısını ölçerken eğitim verisindeki yüksek sonuçlara aldanmamak gerektiğidir. Doğrulama (Validation) eğrilerinin takibi, modelin yeni bir veriyle karşılaştığında nasıl davranacağını anlamak için vazgeçilmezdir. Gelecekteki projelerimde, bu çalışmada elde ettiğim "Temel ML Reçetesi" bilgilerini kullanarak daha dengeli ve genelleme kapasitesi yüksek projeler geliştirmeyi hedefliyorum.

7. Kaynakça

[1] D. Sculley, G. Holt, D. Golovin, E. Davydov, and T. Phillips, "Hidden Technical Debt in Machine Learning Systems.," *Advances in Neural Information Processing Systems (NIPS).* pp. 1–9, 2015.

[2] P. Domingos, "A Few Useful Things to Know About Machine Learning," *Communications of the ACM,* vol. 55, no. 10, pp. 78–87, 2012.

[3] M. Hossin and M. N. Sulaiman, "A Review on Evaluation Metrics for Data Classification Evaluations," *International Journal of Data Mining & Knowledge Management Process (IJDKP),* vol. 5, no. 2, pp. 1–11, 2015.

[4] R. Kohavi, "A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection.," *International Joint Conference on Artificial Intelligence (IJCAI).* 1995.